

模型行为描述

Modelica语法详解

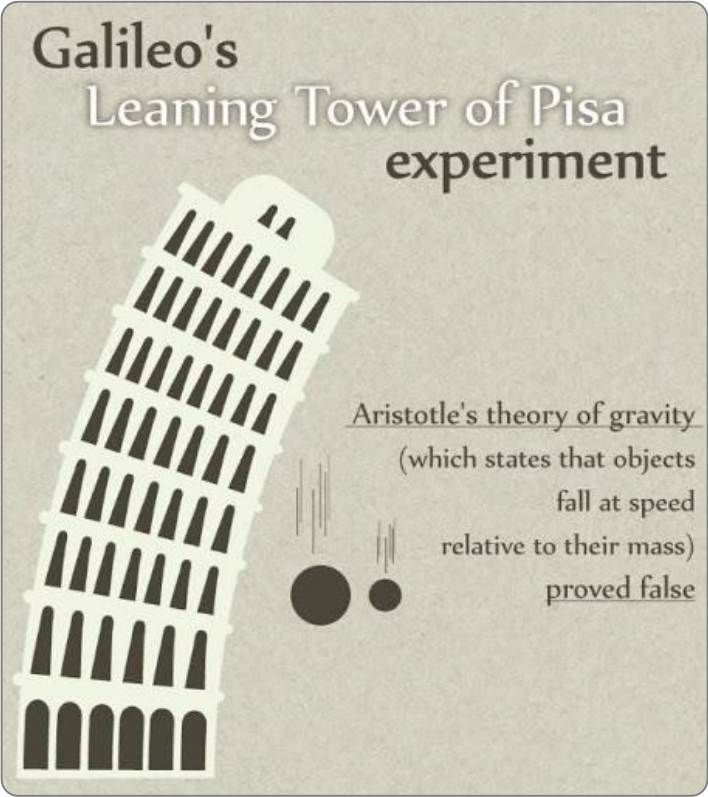
苏州同元软控信息技术有限公司

2024年2月27日



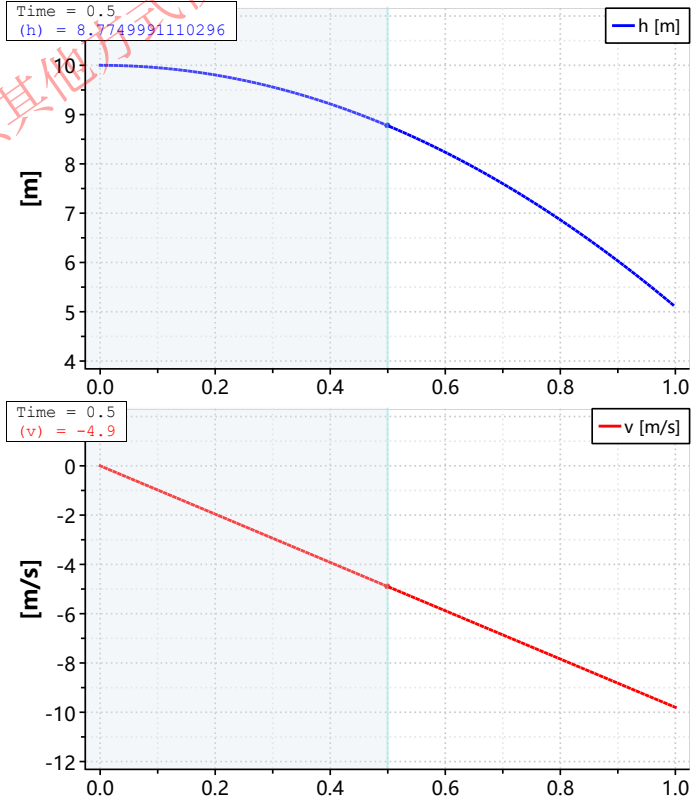
TONGYUAN
Software and Control

Examples



$$\begin{cases} h = v \cdot t \\ a = \dot{v} = -g \end{cases}$$

```
model Fall "自由落体"  
  final parameter Real g = 9.81;  
  Real h(start = 10);  
  Real v;  
  equation  
    v = der(h);  
    der(v) = -g;  
end Fall;
```



如何约束小球的运动？

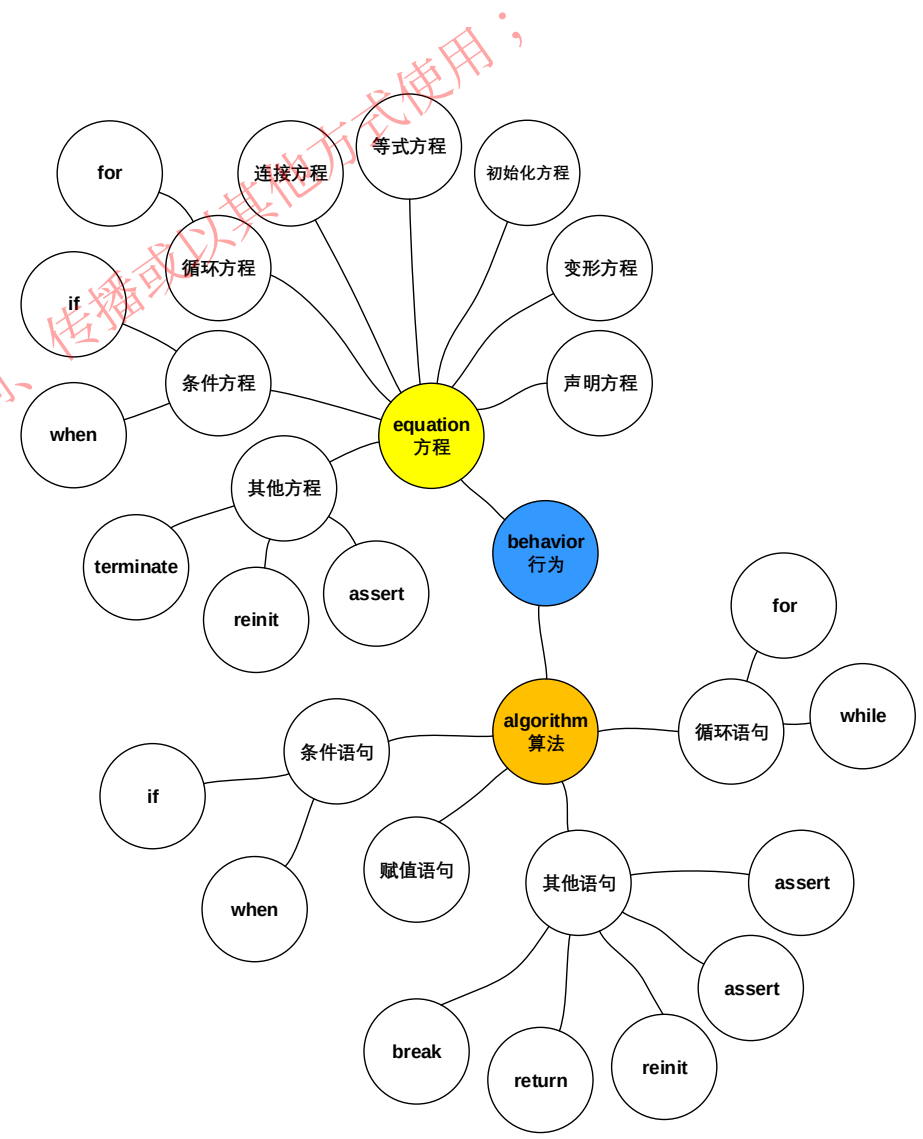
思考

```
class name
  Declaration1
  Declaration2
  ...
equation
  equation1
  equation1
  ...
end name;
```

```
class HelloWorld
  Real x(start = 0);
  parameter Real a = 1;
equation
  der(x) = -a * x;
end HelloWorld;
```

- 类声明，定义模型名称
- 声明参数
 - 声明变量
 - 声明继承或被调用的模型（如接口）
- 通过方程或算法，描述行为
- 结束类定义

构建一个模型，需要定义对象的属性和行为。
前两章已经学习了对象属性的定义，
那么对象的行为该如何定义？



苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其它方式使用；

目录

1. 方程-equation

2. 算法-algorithm

3. 本章回顾

1. 方程-equation

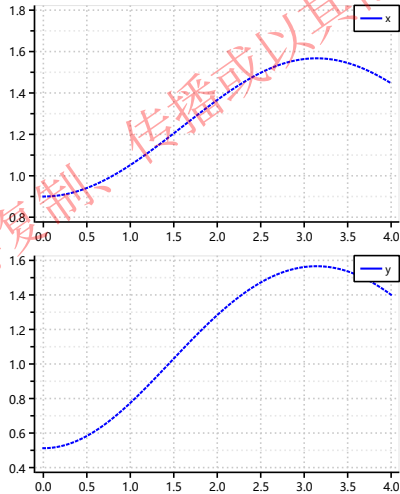
特点：以**陈述式**方程表达模型的行为，模型行为即模型的数学方程或物理方程。

声明区域

```
model DAEexample "DAE系统示例"  
  parameter Real x0 = 0.9;  
  Real x;  
  Real y;
```

方程区域

```
  initial equation  
    x = x0;  
  equation  
    der(y) + (1 + 0.5 * sin(y)) * der(x) = sin(time);  
    x - y = exp(-0.9 * x) * cos(y);  
end DAEexample;
```



注意:
方程区域以 “initial equation” 或 “equation” 关键词开始，终止于类定义结束 “end” 或关键词 “public” 、 “protected” 、 “algorithm” 、 “equation” 、 “initial algorithm” 、 “initial equation” 之一。

方程分类

声明区域方程	声明方程		给定变量约束
	变形方程		替换类的声明方程，用作属性修改
方程区域方程	常规方程	初始化方程	方程区定义的方程， 定义各变量之间的关系。
		等式方程	
		连接方程	
		循环方程	
		条件方程	
		其他方程	



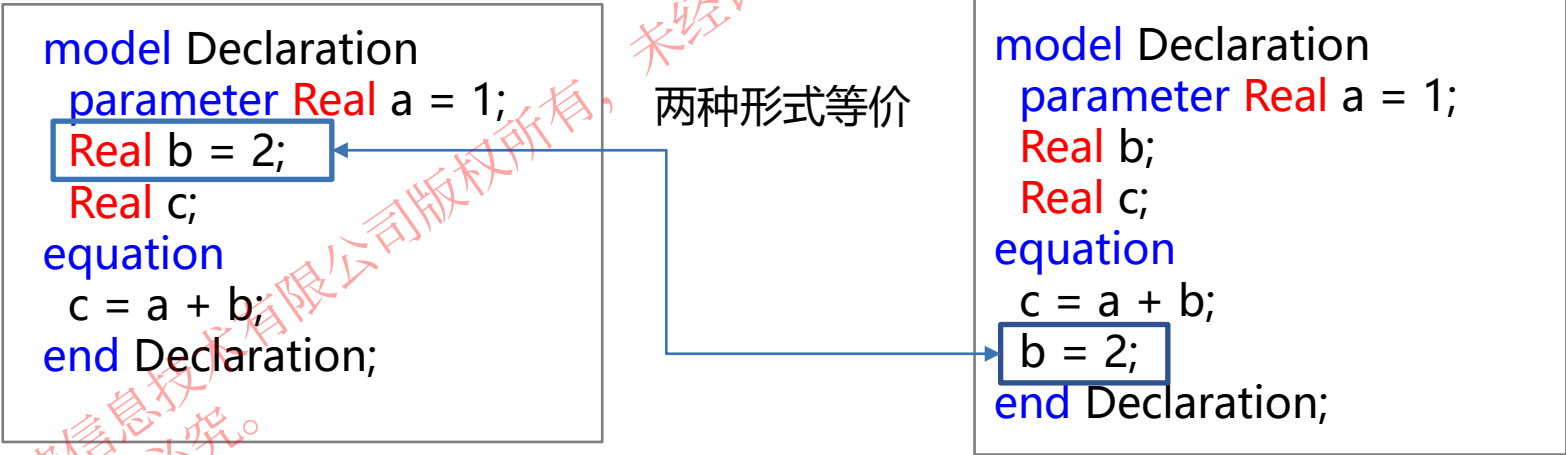
1. 方程-equation



作用： 在变量声明的同时给定变量的约束

一般使用场景

变量取特定值：



注意：

1. 声明方程给定变量的约束，在整个仿真过程中**始终成立**。
2. 一般变量取特定值时使用声明方程的形式，其他不推荐使用。

1. 方程-equation



作用： 修改类的属性，即替换类的声明方程或增加新的方程

一般使用场景

拖拽式建模：



改变参数



组件参数			
常规			
参数			
R	20	Ohm	电阻值

根据参数设定，自动生成变形方程

```
model Deformation
  RLC.BasicModel.Resistance resistance
  annotation (...);
end Deformation;
```

```
model Deformation
  RLC.BasicModel.Resistance resistance(R = 20)
  annotation (...);
end Deformation;
```



1. 方程-equation



作用：定义变量的初始化值

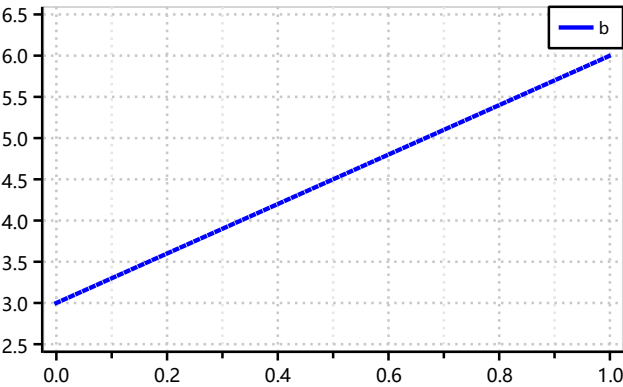
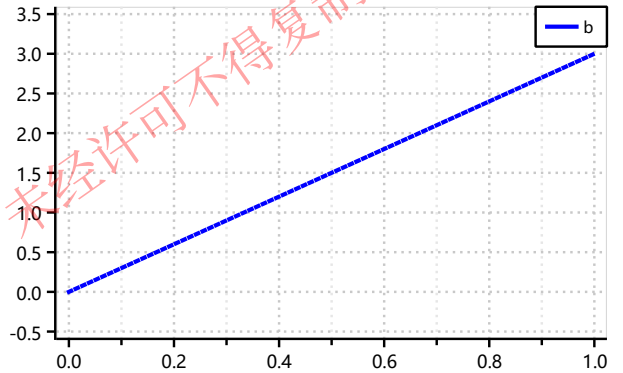
一般使用场景

等式中存在积分环节：

```
model Integral
  parameter Real a = 3;
  Real b;
  equation
    der(b) = a;
end Integral;
```

↓ 设定初始值

```
model Integral
  parameter Real a = 3;
  Real b;
  initial equation
    b = 3;
  equation
    der(b) = a;
end Integral;
```



注意：

- 等式中存在积分环节时，需给定初始值，不给定初始值，则默认为初始值，即0或false
- “initial equation” 与 “start=n, fixed=true” 功能上等价，均为必须满足的初始值



1. 方程-equation



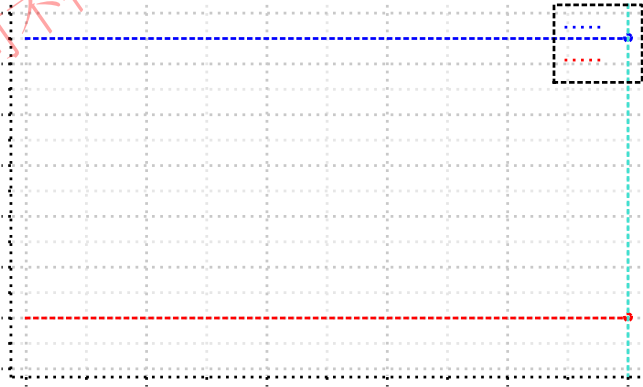
作用： 定义各变量之间的约束关系

一般使用场景

定义等式两边的约束

$$\begin{cases} x_1 + x_2 = 35 \\ 2x_1 + 4x_2 = 94 \end{cases}$$

```
model SimpleMath
  Real x1 ;
  Real x2 ;
  equation
    x1 + x2 = 35;
    2 * x1 + 4 * x2 = 94;
end SimpleMath;
```



注意：

- 无需考虑左右两边的顺序
- 方程左侧不可写if方程
- 只有方程数等于变量数，才可以编译仿真



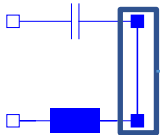
1. 方程-equation



作用：表示组件之间接口的连接

一般使用场景

拖拽式建模中连接组件



自动生成代码

```
model Conection
  RLC.BasicModel.Resistance resistance
    annotation (...);
  RLC.BasicModel.Capacitor capacitor
    annotation (...);
  equation
    connect(capacitor.positivePin, resistance.positivePin)
    annotation (...);
end Conection;
```

形式：concect(接口1， 接口2)

annotation(...); ← 表示连线的显示

注：连接方程一般不需文本书写，组件连接后自动生成



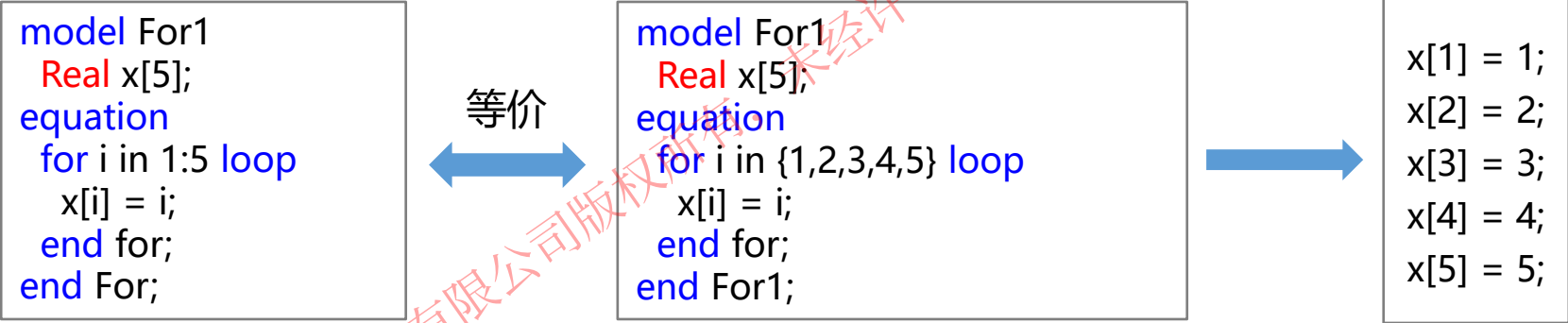
1. 方程-equation



for方程

作用：使循环变量在一定范围内变化，对结构形式相同的方程进行迭代计算。

一般使用场景 单个迭代变量计算



形式:



1. 方程-equation



for方程

作用： 使循环变量在一定范围内变化，对结构形式相同的方程进行迭代计算。

一般使用场景 多个迭代变量计算

```
model For2
  Real x[2,4];
equation
  for i in 1:2 loop
    for j in 1:4 loop
      x[i,j] = i + j;
    end for;
  end for;
end For2;
```

等价

```
model For2
  Real x[2,4];
equation
  for i in 1:2, j in 1:4 loop
    x[i,j] = i + j;
  end for;
end For2;
```



```
x[1,1] = 1+1;
x[1,2] = 1+2;
x[1,3] = 1+3;
x[1,4] = 1+4;
x[2,1] = 2+1;
x[2,2] = 2+2;
x[2,3] = 2+3;
x[2,4] = 2+4;
```

形式:

```
for <var1> in <range1>, <var2> in <range2> loop
  <方程>
end for;
```

注意:

- 1. 使用 “,” 隔开多个迭代器
- 2. range1、range2均为向量



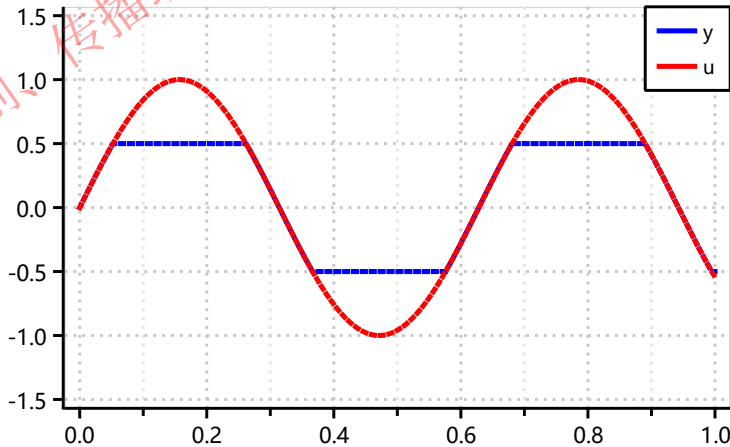
1. 方程-equation



if方程

作用：根据不同的判断条件选择计算方式。

```
model If
  Real u = sin(10 * time);
  Real y;
  equation
    if u > 0.5 then
      y = 0.5;
    elseif u < -0.5 then
      y = -0.5;
    else
      y = u;
    end if;
  end If;
```



形式1:

```
if <条件> then
  <方程>
elseif <条件> then
  <方程>
else
  <方程>
end if;
```

注意:

- 1. 各分支方程数量必须一致;
- 2. 各分支的条件均为布尔量;
- 3. elseif 可出现0到多次;
- 4. else最多出现一次, 如果if和elseif分支的条件为参数或常量, 则可以没有else分支;



1. 方程-equation



if方程

作用：根据不同的判断条件选择计算方式。

简写方式：用于分支方程数量为1的简单if语句赋值;

```
model If
  Real u = sin(10 * time);
  Real y;
  equation
    y = if u>0.5 then 0.5 elseif u<-0.5 then -0.5 else u;
  end If;
```

形式2:

```
<variable> = if <条件1> then <value1> else if <条件2> then <value2> else
<value3>
```



1. 方程-equation



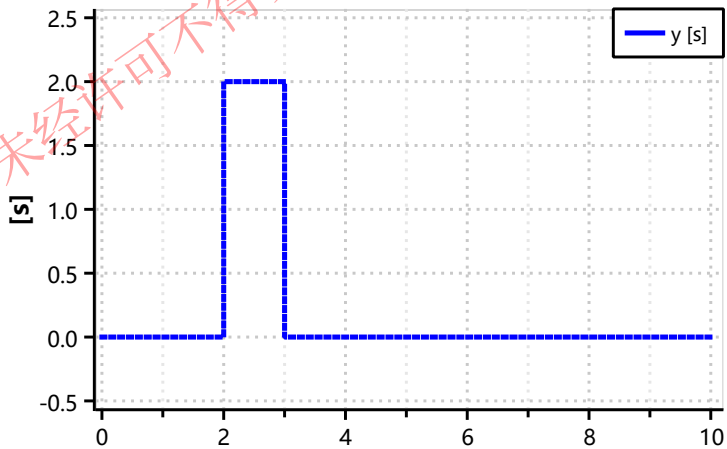
when方程

作用：表示在事件时刻有效的瞬态方程, 条件变为true时触发一次。

```
model When
  Real x = time;
  Real y;
  equation
    when x > 2 then
      y = x;
    elseif x > 3 then
      y = 0;
    end when;
end When;
```

形式:

```
when <条件> then
  <方程>
elseif <条件> then
  <方程>
end when ;
```



注意:

- 1. when语句中左边变量为离散变量
- 2. elseif可以出现0到多次
- 3. when方程不能嵌套在when、if、for方程中。

<方程>只能是以下形式之一

- 1. y = expr, 左边是变量名
- 2. (y1, y2, ...) = function(in1, in2, ...), 左边为变量列表
- 3. assert(), terminate(), reinit()
- 4. 满足上述要求的if方程和for方程, 不能有when方程。



1. 方程-equation

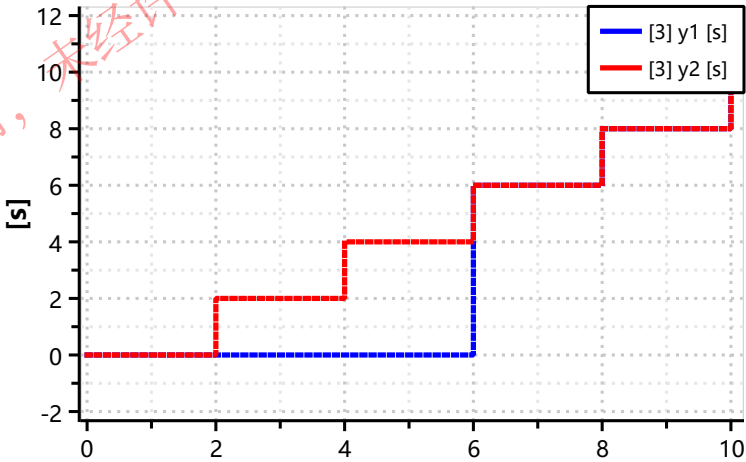


when方程

注意事项

- 1. <条件>是布尔型的标量或向量，
标量条件变为true，该分支的方程生效
向量条件中只要任何一个元素变为true，该分支中的方程就生效；

```
model When
  Real x = time;
  Real y1;
  Real y2;
  equation
    when sample(0, 2) or x < 5
    then
      y1 = x;
    end when;
    when {sample(0, 2), x < 5}
    then
      y2 = x;
    end when;
  end When;
```



如果改成{sample(0, 2), x > 5}, 结果会有什么变化?



1. 方程-equation

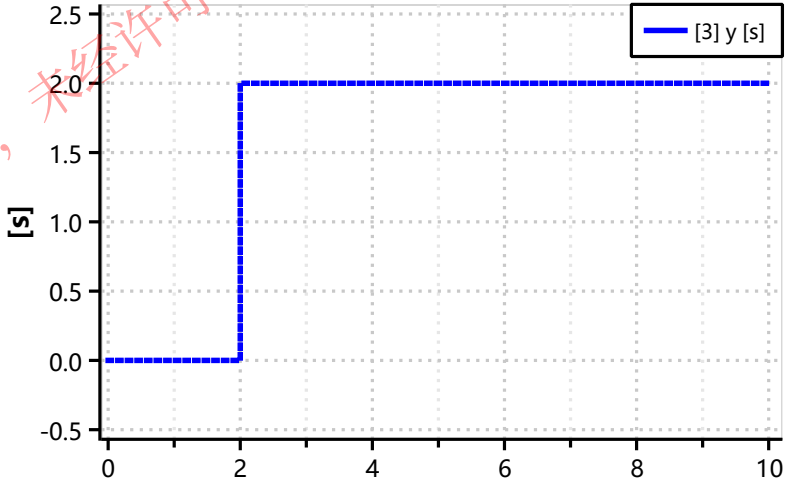


when方程

注意事项

2. when中方程优先级高于elsewhen中方程;

```
model When
  Real x = time;
  Real y;
  equation
    when time > 2 then
      y = x;
    elseif time > 2
    then
      y = 0;
    end when;
  end When;
```



1. 方程-equation

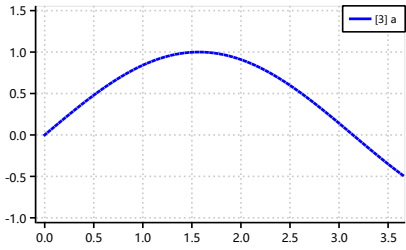
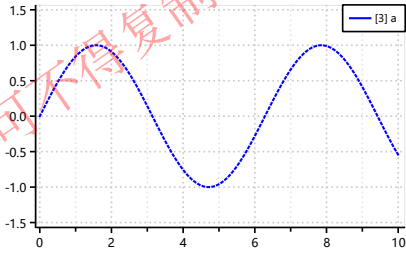


assert方程

作用：模型检查和校验的一种手段，当条件不满足时，输出消息，停止仿真

```
model Assert
  Real a;
  equation
    a = sin(time);
    assert(a > -1, "退出仿真");
  end Assert;
```

```
model Assert
  Real a;
  equation
    a = sin(time);
    assert(a > -0.5, "退出仿真");
  end Assert;
```



建模 仿真

This log was created by MWSolver at Tue May 04 15:24:39 2021.
MWSolver started ...
An event was occurred at Time = 0
Event processing was done at Time = 0

Simulation started at Time = 0 using integration method dassl
Error:Assert was triggered:
退出仿真
Error:Assert condition:
a > (-0.5)
Simulation terminated at Time = 3.68 (StopTime = 10)
CPU Time for Simulation: 0.001s
Number of Time Events: 0
Number of State Events: 0
Number of Grid Points: 184
Minimum integration stepsize: 0
Maximum integration stepsize: 0.02

错误代码: 5。

形式:

```
assert(<条件>, <消息>)
```

注意:

- 1. <条件>为布尔型;
- 2. <消息>为字符串型, 即 “输出提示”



1. 方程-equation



terminate方程

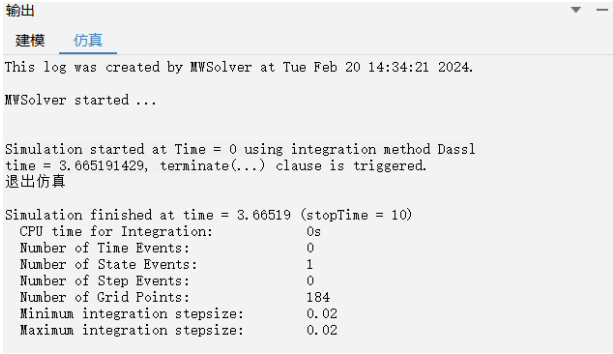
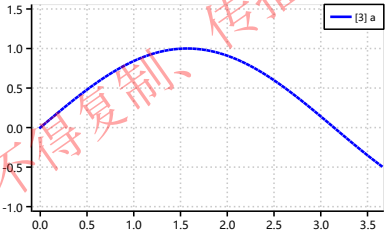
作用： 正常结束仿真程序。

一般与when语句联用，通过when语句触发terminate，仿真模型程序输出指定的消息字符串之后退出

```
model Terminate
  Real a;
  equation
    a = sin(time);
  when a <= -0.5 then
    terminate("退出仿真");
  end when;
end Terminate;
```

形式：

```
terminate(<消息>)
```



注意：
<消息>为字符串型，即“输出提示”



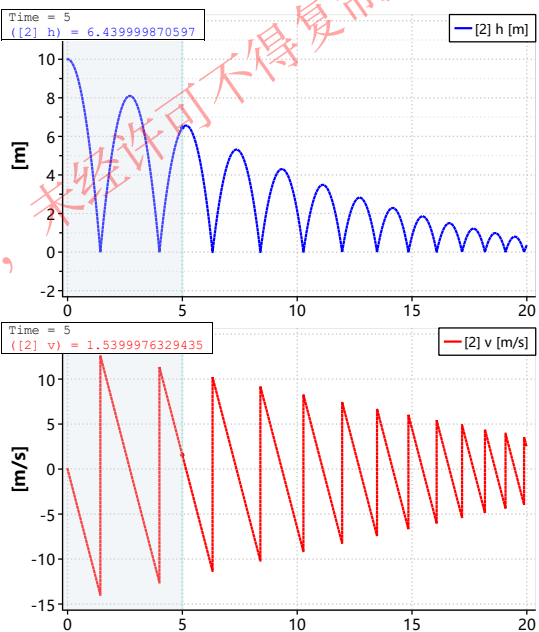
1. 方程-equation



reinit方程

作用：用于重新初始化状态变量

```
model Ball "弹跳小球"  
  final parameter Real g = 9.8 "重力加速度";  
  parameter Real coef = 0.9 "弹性系数";  
  parameter Real h0 = 10 "初始高度";  
  Real h "小球高度";  
  Real v "小球速度";  
  Boolean flying "是否运动";  
  initial equation  
    h = h0;  
  equation  
    flying = not (h <= 0 and v <= 0);  
    der(v) = if flying then -g else 0;  
    v = der(h);  
    when h <= 0 then  
      reinit(v, -coef * v);  
    end when;  
  end Ball;
```



- 注意:**
- 1. reinit只能使用在when语句中
 - 2. 同一个状态变量只能在一个方程中使用reinit

形式:

reinit(状态变量, 新的初始值)



1. 方程-equation

声明区域方程	声明方程			给定变量约束
	变形方程			替换类的声明方程，用作属性修改
方程区域方程	初始化方程		initial equation	用作变量的初始化定义
	常规方程	等式方程	=	定义变量直接的关系
		连接方程	connect	连接组件
		循环方程	for	用于多个迭代变量计算
		条件方程	if	用于多种计算方式的选择
			when	用于表示在事件时刻有效的瞬态
		其他方程	assert()	用于模型检查与校验
			terminate	用于正常结束仿真程序
			reinit()	用于重新初始化状态变量



目录

1. 方程-equation

2. 算法-algorithm

3. 本章回顾

2. 算法 - algorithm

方程与算法的区别：

方程采用陈述式建模，即不指定数据流向和控制流，等式赋值没有顺序；
算法采用过程式建模，即语句按其出现的顺序执行，且等号左边是未知量，
右边是已知量；

```
model Average
  parameter Real x[:] = {10, 20, 30, 40, 50};
  Real average;
  Real sum;
  algorithm
    sum := 0;
    for i in 1:size(x, 1) loop
      sum := sum + x[i];
    end for;
    average := sum / size(x, 1);
  end Average;
```

注意：
尽量减少使用algorithm，能用equation尽量用equation。
算法区域作为一个整体，因此一般将算法封装成function。

```
model Average
  parameter Real x[:] = {10, 20, 30, 40, 50};
  Real average;
  equation
    average = ModelicaGrammar.Behavior.Average_f(x);
  end Average;
```

```
function Average_f
  input Real x[:];
  output Real average;
  Real sum;
  algorithm
    sum := 0;
    for i in 1:size(x, 1) loop
      sum := sum + x[i];
    end for;
    average := sum / size(x, 1);
  end Average_f;
```

函数调用

算法区域以“equation” 关键词开始，终止于类定义结束 “end” 或关键词 “public” 、 “protected” 、 “algorithm” 、 “equation” 、 “initial algorithm” 、 “initial equation” 之一。
算法只能出现在算法区域。

2. 算法-algorithm

算法由一系列算法语句组成。

算法语句	赋值语句	“:=”	定义各变量之间的约束关系
	循环语句	for	用于迭代变量计算(与等式中使用相同)
		while	用于具有约束条件的迭代计算
	条件语句	if	用于多种计算方式的选择(与等式中使用相同)
		when	用于表示在事件时刻有效的瞬态(与等式中使用相同) 不能用于function中
	其他语句	break	用于终止for、while循环计算
		return	终止函数调用，返回当前输出变量的值
		assert	用于模型检查与校验(与等式中使用相同)
		terminate	用于正常结束仿真程序(与等式中使用相同)
		reinit	用于重新初始化状态变量(与等式中使用相同)

2. 算法-algorithm

赋值语句

循环语句

条件语句

其他语句

作用： 定义各变量之间的约束关系

```
function Average_f
  input Real x[:];
  output Real average;
  Real sum;
algorithm
  sum := 0;
  for i in 1:size(x, 1) loop
    sum := sum + x[i];
  end for;
  average := sum / size(x, 1);
end Average_f;
```

形式：

```
a := b
```

使用 “:=” 区分与 “=” 的含义不同



2. 算法-algorithm

赋值语句

循环语句

条件语句

其他语句

for语句

作用：使循环变量在一定范围的值里变化，对结构形式相同的方程进行迭代计算。

```
function Average_f
  input Real x[:];
  output Real average;
  Real sum;
algorithm
  sum := 0;
  for i in 1:size(x, 1) loop
    sum := sum + x[i];
  end for;
  average := sum / size(x, 1);
end Average_f;
```

使用方式与等式中for方程完全相同，不再赘叙。

2. 算法-algorithm

赋值语句

循环语句

条件语句

其他语句

while语句

作用：用于约束条件的迭代计算

```
function Average_f
input Real x[:];
output Real average;
Real sum;
Integer i;
algorithm
sum := 0;
i := 0;
while i <= size(x, 1) loop
  i = i + 1;
  sum := sum + x[i];
end while;
average := sum / size(x, 1);
end Average_f;
```

形式:

```
while <条件> loop
  <语句>
end while
```

for语句用于已知迭代次数的算法
while语句用于已知需满足的条件，不限迭代次数的算法

- 条件为布尔量
- 条件的值为true，则进入循环
- 条件的值为false，则转到“end while”之后执行

2. 算法-algorithm

赋值语句

循环语句

条件语句

其他语句

if语句

作用：根据不同的判断条件选择计算方式。

```
function Average_f
input Real x[:];
output Real average;
algorithm
average := 0;
for i in 1:size(x, 1) loop
if x[i] > 0 then
average := average + x[i];
else
average := average - x[i];
end if;
end for;
average := average / size(x, 1);
end Average_f;
```

通过if语句判断，求解输入向量中所有值的绝对值的平均数

使用方式与等式中if方程完全相同，不再赘叙。

2. 算法-algorithm

赋值语句

循环语句

条件语句

其他语句

when语句

作用：表示在事件时刻有效的瞬态方程, 条件变为true时触发一次。

```
model When
  Real x;
  Real y;
  algorithm
    when x > 2 then
      y = x;
    elseif x > 3 then
      y = 0;
    end when;
end When;
```

不能用于function中，只能用于model或block中。

使用方式与等式中when方程完全相同，不再赘叙。

2. 算法-algorithm



break语句

作用： 用于终止while/for循环语句

```
function Position
input Real x[:];
input Real val;
output Integer index;
algorithm
index := size(x, 1);
while index >= 1 loop
    if x[index] == val then
        break;
    else
        index := index - 1;
    end if;
end while;
end Position;
```

while/for循环中遇到break语句:
转到最内层的 “end while” / “end while” 后
执行。

思考：
如果删除break结果会如何？

注意：
break语句只能用在算法中的while/for循环语句中。

2. 算法-algorithm

赋值语句

循环语句

条件语句

其他语句

return语句

作用： 用于终止函数调用，输出变量的当前值作为函数调用的结果返回

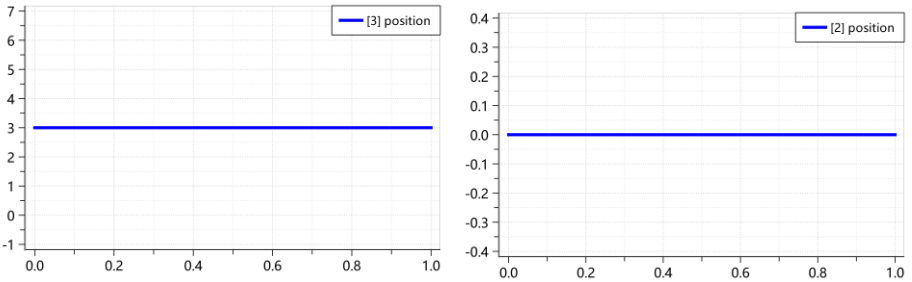
```
function Position
input Real x[:];
input Real val;
output Integer index;
algorithm
for i in 1:size(x, 1) loop
if x[i] == val then
index := i;
return;
end if;
end for;
index := 0;
end findValue;
```

```
model Location
parameter Real a[5] = {1, 4, 6, 7, 3};
parameter Real b = 6;
Real position;
equation
position = ModelicaGrammar.Behavior.Position(a, b);
end Location;
```

思考：

如果删除return结果会如何？

如果将return改成break会如何？



注意：

return语句只能在function中使用。



2. 算法-algorithm

赋值语句

循环语句

条件语句

其他语句

assert语句

作用：模型检查和校验的一种手段。

terminate语句

作用：正常结束仿真程序。

reinit语句

作用：用于重新初始化状态变量(应用了der()的变量)。

使用方式与等式中assert、terminate、reinit方程完全相同，不再赘叙。

2. 算法-algorithm

小结

算法语句	赋值语句	“:=”	定义各变量之间的约束关系
	循环语句	for	用于多个迭代变量计算(与等式中使用相同)
		while	用于具有约束条件的迭代计算
	条件语句	if	用于多种计算方式的选择(与等式中使用相同)
		when	用于表示在事件时刻有效的瞬态(与等式中使用相同) 不能用于function中
	其他语句	break	用于终止for、while循环计算
		return	终止函数调用，返回当前输出变量的值
		assert	用于模型检查与校验(与等式中使用相同)
		terminate	用于正常结束仿真程序(与等式中使用相同)
		reinit	用于重新初始化状态变量(与等式中使用相同) 不能用于function中

目录

1. 方程-equation

2. 算法-algorithm

3. 本章回顾

3. 本章回顾

声明区域方程	声明方程			给定变量约束
	变形方程			替换类的声明方程，用作属性修改
方程区域方程	初始化方程		initial equation	用作变量的初始化定义
	常规方程	等式方程	=	定义变量直接的关系
		连接方程	connect	连接组件
		循环方程	for	用于多个迭代变量计算
		条件方程	if	用于多种计算方式的选择
			when	用于表示在事件时刻有效的瞬态
		其他方程	assert()	用于模型检查与校验
			terminate	用于正常结束仿真程序
			reinit()	用于重新初始化状态变量
算法区语句	算法语句	赋值语句	“:=”	定义各变量之间的约束关系
		循环语句	for	用于多个迭代变量计算(与等式中使用相同)
			while	用于具有约束条件的迭代计算
		条件语句	if	用于多种计算方式的选择(与等式中使用相同)
			when	用于表示在事件时刻有效的瞬态(与等式中使用相同) 不能用于function中
		其他语句	break	用于终止for、while循环计算
			return	终止函数调用，返回当前输出变量的值
			assert	用于模型检查与校验(与等式中使用相同)
			terminate	用于正常结束仿真程序(与等式中使用相同)
			reinit	用于重新初始化状态变量(与等式中使用相同)

3. 本章回顾

课堂回顾

1. 下面属于声明方程的是 () 。
A. Real v=100 B. $R*I=V$ C. $V:=10$ D. if方程
2. 和方程 $R*I=V$ 等价的是 () 。
A. $V:=R*I$ B. $R:=V/I$ C. $I:=V/R$ D. $I=V/R$
3. 可以表达循环的方程是 () 。
A. 声明方程 B. 等式方程 C. for方程 D. if方程
4. 用来做判断的方程是 () 。
A. 变型方程 B. for方程 C. if方程 D. when方程
5. if方程中else分支最多可以出现 () 次 。
A. 1 B. 2 C. 3 D. 无限制
6. when方程中可以嵌套 () 个when方程 。
A. 0 B. 1 C. 2 D. 无限制
7. 给用户提供错误提示, 并终止仿真的是 () 。
A. if B. when C. assert D. reinit

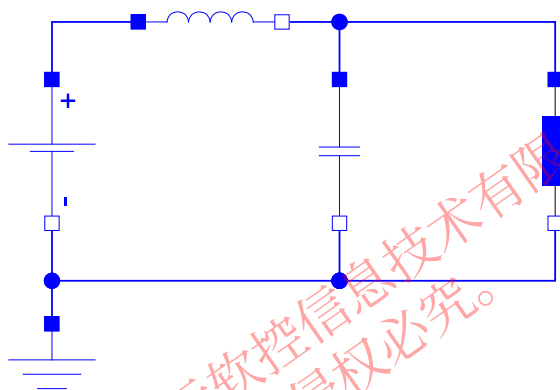
8. 用于状态变量初始化的方程是 () 。
A. if B. when C. assert D. reinit
9. 算法区域以下列关键字 () 开始。
A. if B. when C. equation D. algorithm
10. 可以中断循环的是 () 。
A. for B. while C. if D. break
11. 正常终止仿真, 并输出终止原因的是 () 方程。
12. 可以用于连接的是 () 方程 。
13. 条件有false变为true的瞬间, 其中的方程计算一次, 该
方
程是 () 方程 。
14. when语句中不能嵌套在 () 语句中。
15. break语句只能用于算法段中 () 语句或是 () 语句 。

3. 本章回顾

课后作业

1. 使用算法，采用for语句和while语句两种方式，定义一个“n!”阶乘模型，结果可根据n的值进行计算。
2. 使用等式，采用for方程的方式，定义“n!”阶乘模型，并将1!, 2!, 3!, ..., n!均存储至结果数列中。
3. 已知RLC电路两端电压值为24V，利用Modelica语法根据物理拓扑关系描绘出仿真模型，并观测电容、电感两端电压以及流过的电流的变化。

RLC电路物理拓扑图以及系统原理方程如下：



$$V_R = i_R \cdot R$$

$$C \cdot \frac{dV_C}{dt} = i_C$$

$$L \cdot \frac{di_L}{dt} = V_L$$

$$V_R = V_C = V - V_L$$

$$i_L = i_R + i_C$$

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

谢谢！